

Output Parsers in LangChain

1. Recap (from previous video: Structured Output)

- LLMs (like GPT, Llama) usually return **unstructured text**.
- But other systems (like databases, APIs) need **structured data** (JSON, tables).
- **Structured Output** forces LLMs to return in a fixed format.
- Some LLMs:
 -  *Can give structured output* (like GPT models).
 -  *Cannot give structured output* (most open-source models).

👉 For “cannot” models, we use **Output Parsers**.

2. What are Output Parsers?

- Output Parsers = Tools in LangChain that **convert raw LLM responses (text)** into structured formats like JSON, Pydantic, CSV, etc.
- Benefits:
 -  Consistency → output always in same format
 -  Easy to use in apps
 -  Can enforce structure (schema)
 -  Can even validate data (with Pydantic)

👉 They work with **both kinds of models**:

- Can (GPT etc.)
 - Cannot (open-source like HuggingFace models)
-

3. Types of Output Parsers

In LangChain, there are many, but the **4 most important** are:

1. **StringOutputParser**
 2. **JsonOutputParser**
 3. **StructuredOutputParser**
 4. **PydanticOutputParser**
-

4. Detailed Study of Each Parser

◆ 4.1 String Output Parser

- **Simplest parser.**
- Just extracts **text response** from LLM and returns it as a plain string.
- Normally, LLM response contains extra metadata (tokens, usage, etc.).
 - Without parser → you use `result.content`.
 - With parser → directly get clean text.

Use Case:

- When chaining multiple prompts.
- Example:
 1. First prompt → generate detailed report.
 2. Second prompt → summarize that report in 5 lines.
- String Parser extracts only text and passes it to the next step.

 Works well with **chains** → makes pipelines clean.

◆ 4.2 JSON Output Parser

- Forces LLM to return in **JSON format**.
- Quickest way to get JSON.

Steps:

1. Create a `JsonOutputParser`.
2. Add its `get_format_instructions()` to your prompt.
3. Model will return JSON.

4. Use `parser.parse(result.content)` → Python dictionary.

Example:

```
{
  "name": "John",
  "age": 25,
  "city": "Delhi"
}
```

✅ Output is usable directly in code.

❌ Problem → **No schema enforcement.**

- LLM decides the structure.
- Example: You want "fact1", "fact2", "fact3".
- LLM may just give "facts": [...].

◆ 4.3 Structured Output Parser

- Similar to JSON parser but with **schema enforcement**.
- You define **ResponseSchema** (what keys should exist).
- LLM is forced to return data in that structure.

Example Schema:

- fact1: first fact about topic
- fact2: second fact
- fact3: third fact

👉 Model will return JSON with exactly these keys.

✅ Advantage → structure is **guaranteed**.

❌ Limitation → **No data validation.**

- Example: If you expect "age": int, model may still return "35 years" (string).

◆ 4.4 Pydantic Output Parser

- Most powerful parser.
- Uses **Pydantic models** for schema + validation.
- Can enforce:
 - Data types (string, int, list)
 - Constraints (e.g., age > 18)
 - Descriptions

Example:

python

📄 Copy code

```
class Person(BaseModel):
    name: str
    age: int = Field(gt=18)
    city: str
```

✅ Advantages:

- ✓ Schema enforcement
- ✓ Data validation
- ✓ Type safety
- ✓ Works easily with chains

This is the **best parser** when you need reliable production-level output.

5. Comparison of Parsers

Parser	What it does	Advantage	Limitation
StringOutputParser	Converts response → plain string	Simple, good for chains	Only text
JsonOutputParser	Forces JSON output	Easy JSON parsing	No schema
StructuredOutputParser	Enforces schema (keys, structure)	Consistent structure	No validation
PydanticOutputParser	Schema + validation	Reliable, production-ready	Slightly complex setup

6. Other Parsers (in docs)

- CSV Output Parser
- List Output Parser
- Markdown Parser
- Enum Parser
- DateTime Parser
- Output Fixing Parser (if response wrong, fixes it automatically)

👉 But the **main 4** cover 80% use cases.

7. Key Takeaways

- Output Parsers = bridge between **raw LLM output** and **usable structured data**.
- Always use them in real apps → avoid messy unpredictable text.
- Best practice:
 - For text → StringOutputParser
 - For JSON (quick) → JsonOutputParser
 - For structured schema → StructuredOutputParser
 - For production validation → PydanticOutputParser

🎨 Story-Style Understanding

Imagine an **exam**:

1. Student writes **essay freely** → messy (like raw LLM output).
2. Teacher gives a **form** with blank spaces → student fills (Structured Output).
3. But sometimes, student writes in wrong format: "35 years" instead of 35.

👉 Here's how teachers (parsers) help:

- **String Parser** → just copies answer text.
- **JSON Parser** → tells student: "Answer in JSON."
- **Structured Parser** → gives form with exact fields (fact1, fact2, fact3).
- **Pydantic Parser** → checks if the answer is correct type (age must be number).